

ML2 2.4

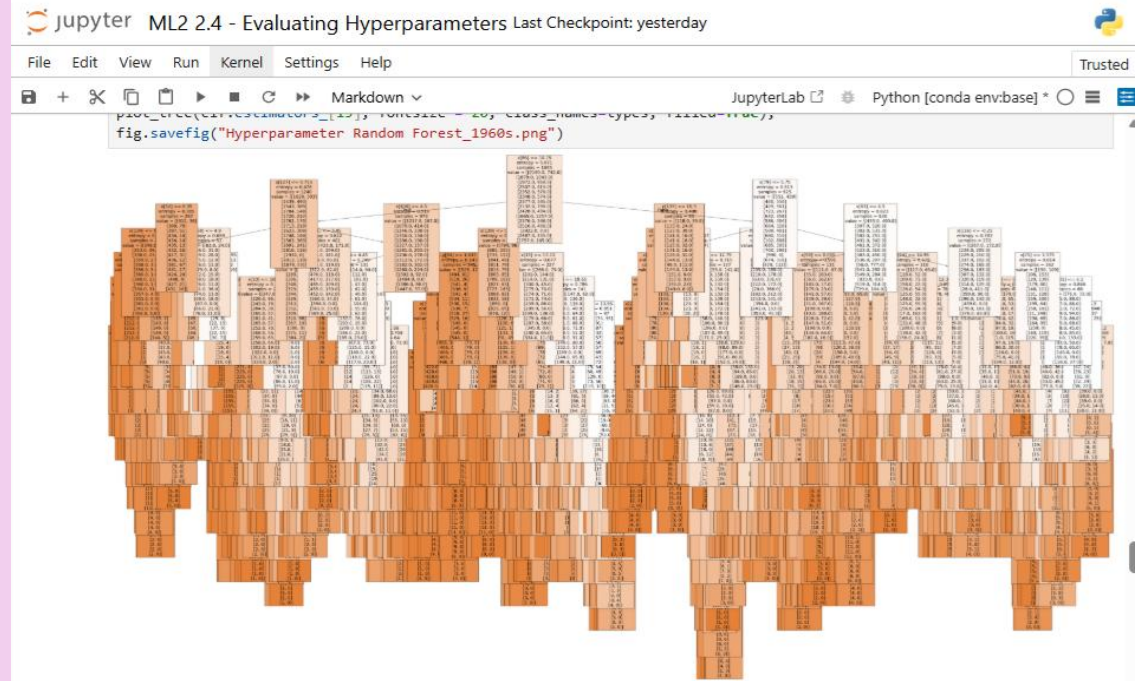
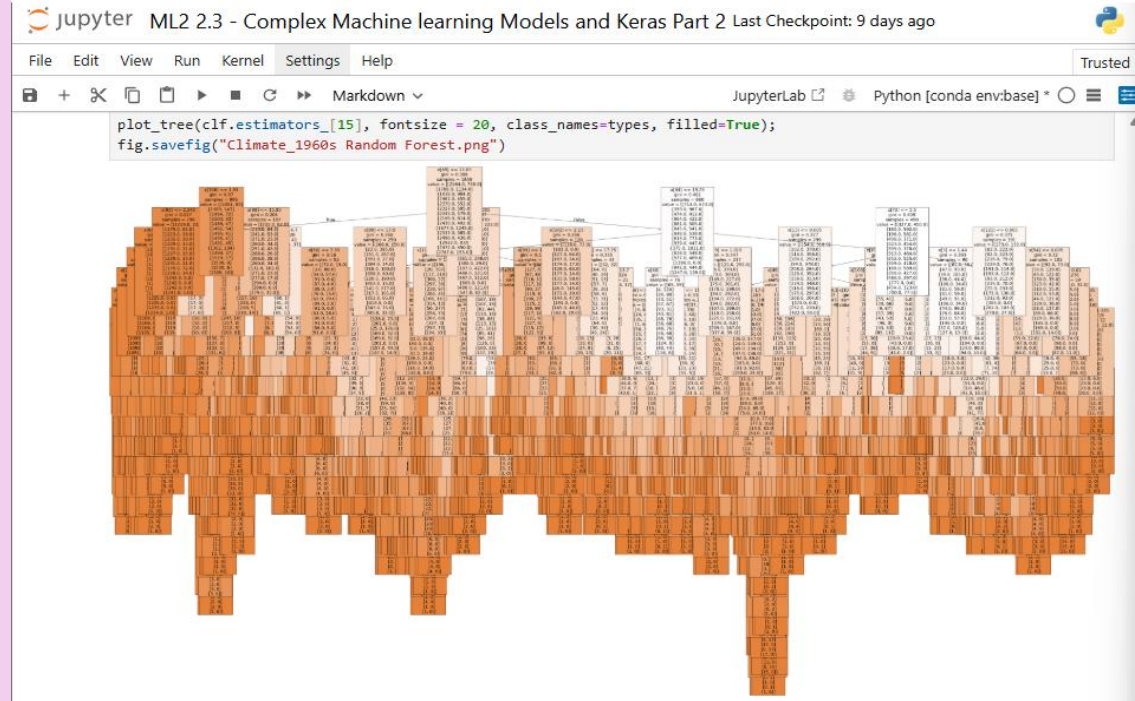
Evaluating Hyperparameters

- Random Forest
- Deep Learning (RNN)

By: Jordan Novelli

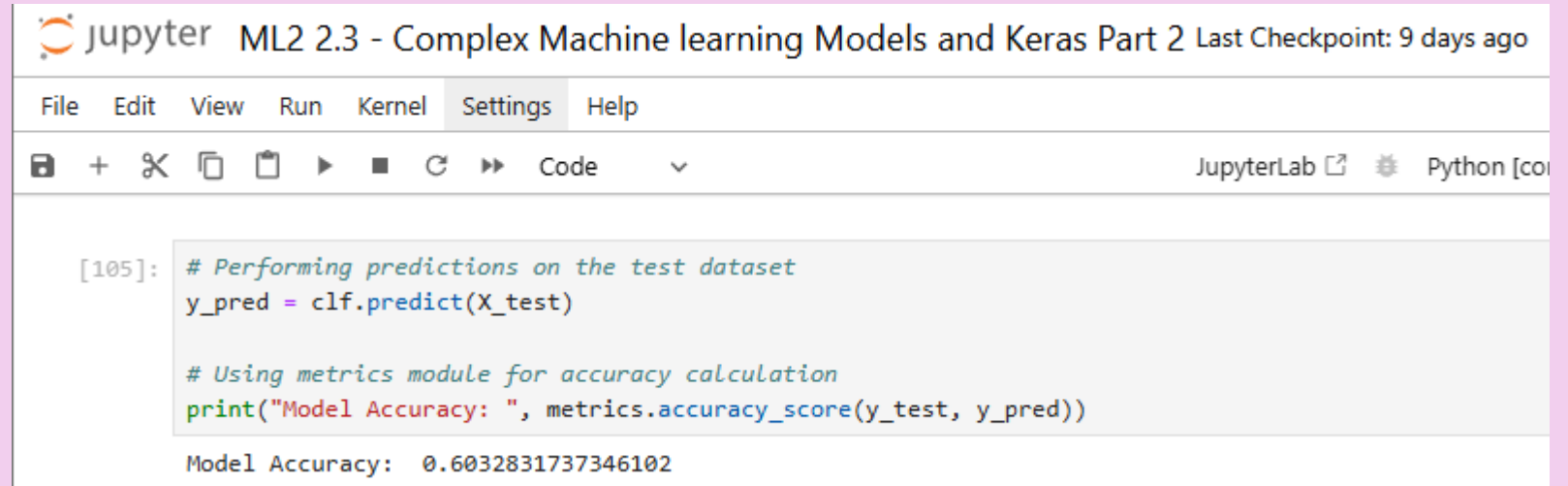
Random Forest with Hyperparameters

- There's not a huge difference in the actual Random Forest images containing the 1960's decade.
- The main difference comes from the accuracy.



Random Forest with Hyperparameters

- Accuracy for the Random Forest was 0.603
- Accuracy for the optimized Random Forest was 0.487.
- That's 0.116 lower than the original.
- Furthermore, the important features for the RF of 1960s continues to be temp_max and temp_mean.



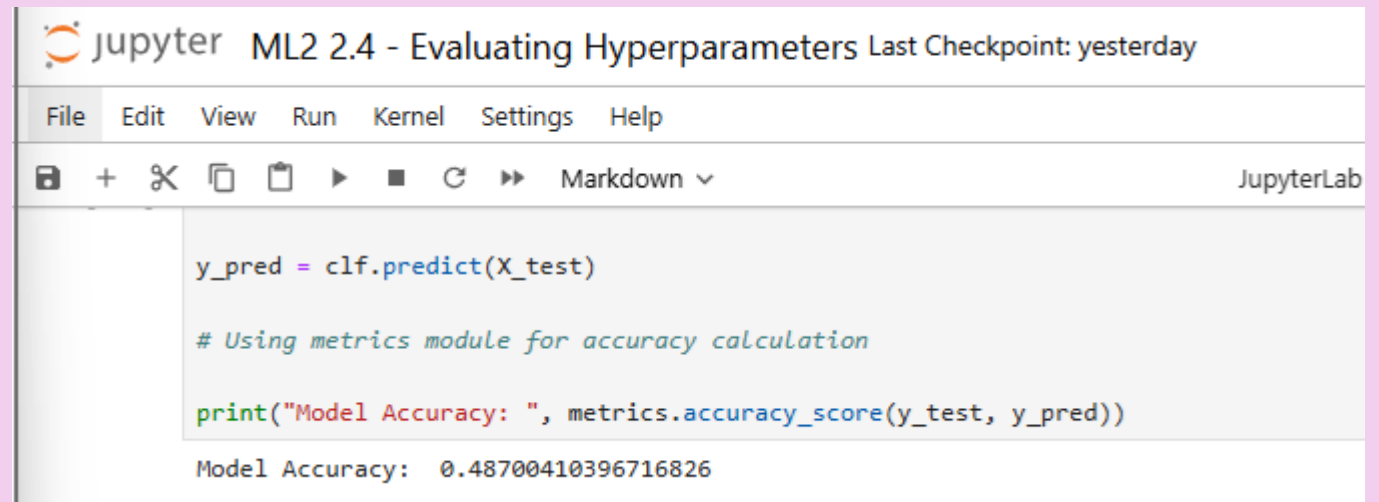
```
jupyter ML2 2.3 - Complex Machine learning Models and Keras Part 2 Last Checkpoint: 9 days ago
```

```
File Edit View Run Kernel Settings Help
```

```
[105]: # Performing predictions on the test dataset
y_pred = clf.predict(X_test)

# Using metrics module for accuracy calculation
print("Model Accuracy: ", metrics.accuracy_score(y_test, y_pred))

Model Accuracy: 0.6032831737346102
```



```
jupyter ML2 2.4 - Evaluating Hyperparameters Last Checkpoint: yesterday
```

```
File Edit View Run Kernel Settings Help
```

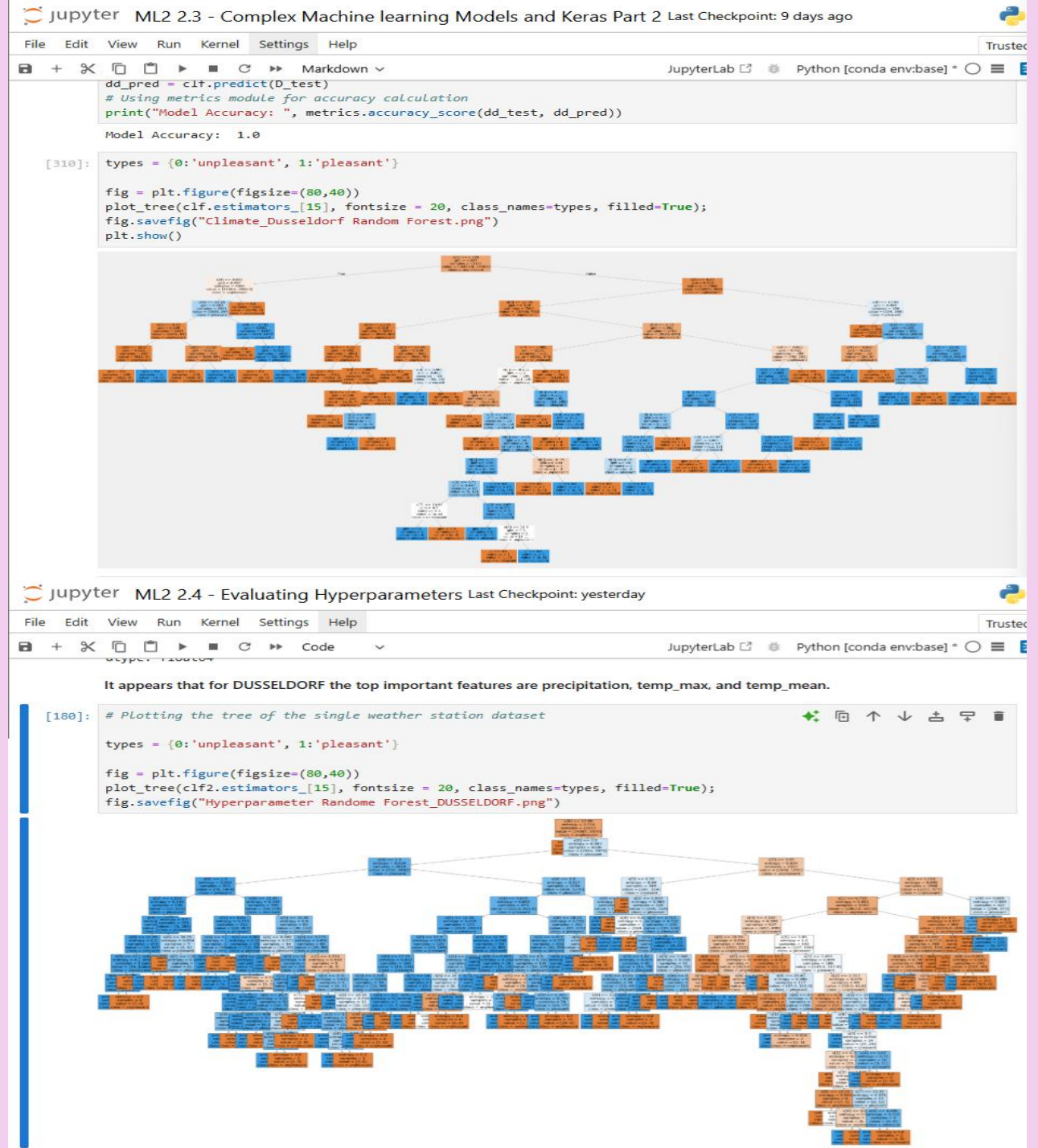
```
y_pred = clf.predict(X_test)

# Using metrics module for accuracy calculation
print("Model Accuracy: ", metrics.accuracy_score(y_test, y_pred))

Model Accuracy: 0.48700410396716826
```

Random Forest with Hyperparameters: DUSSELDORF

- The Random Forest image of the original vs the optimized is largely different.
- The original RF is much more sparse than the optimized RF.
- Accuracy for both the original and optimized RF for DUSSELDORF were 1.0. I believe this is due to the fact that we only looking at the weather station specifically for DUSSELDORF over all the years documented.
- The feature importance for the DUSSELDORF Rfcontinue to be precipitation, temp_max and temp_mean.



Random Forest Analysis Importance

- There isn't much significance when analyzing the original RF and the optimized RF. The original RF created more accuracy when compared to the optimized RF.
- The variables used to determine the RF were no different.



Deep Learning with Hyperparameters

- There is a large difference in the Sequential Model.
- The model on top is the third RNN run in task 2.2.
- The model on bottom is the RNN that was run after using the Bayesian Search in task 2.4.
- In task 2.2 the 3rd RNN sequential model was yielding 10:

Total params: 9,549 (37.30 KB)

Trainable params: 9,549 (37.30 KB)

Non-trainable params: 0 (0.00 B)

- In this current task 2.4 our sequential model was yielding 172:

Total params: 15,283 (59.70 KB)

Trainable params: 15,195 (59.36 KB)

Non-trainable params: 88 (352.00 B)

Layer (type)	Output Shape	Param #
lstm_9 (LSTM)	(None, 15, 44)	9,504
dropout_9 (Dropout)	(None, 15, 44)	0
time_distributed_2 (TimeDistributed)	(None, 15, 1)	45

Total params: 9,549 (37.30 KB)

Trainable params: 9,549 (37.30 KB)

Non-trainable params: 0 (0.00 B)

Model: "sequential_172"

Layer (type)	Output Shape	Param #
conv1d_168 (Conv1D)	(None, 13, 44)	1,232
batch_normalization_94 (BatchNormalization)	(None, 13, 44)	176
dense_854 (Dense)	(None, 13, 44)	1,980
dense_855 (Dense)	(None, 13, 44)	1,980
dense_856 (Dense)	(None, 13, 44)	1,980
dense_857 (Dense)	(None, 13, 44)	1,980
dense_858 (Dense)	(None, 13, 44)	1,980
max_pooling1d_164 (MaxPooling1D)	(None, 6, 44)	0
flatten_164 (Flatten)	(None, 264)	0
dense_859 (Dense)	(None, 15)	3,975

Total params: 15,283 (59.70 KB)

Trainable params: 15,195 (59.36 KB)

Non-trainable params: 88 (352.00 B)

Deep Learning with Hyperparameters

- The model.fit of the 3rd RNN of task 2.2 had an epoch of 30 and an accuracy that started at 0.82 and increased to 0.96.
- In task 2.4 the model.fit had an epoch of 25 and the accuracy started at 0.63 and only increase to 0.64. That wasn't a very good improvement using the Bayesian Search.
- There were some issues with the bayesian search indicating NaN were present when non were showing. This indicates that the algorithms isn't running properly but there are no NaN to be found.

jupyter ML2 2.2 - Complex Machine Learning models & Keras Part 1 Last Checkpoint: 12 days ago		jupyter ML2 2.4 - Evaluating Hyperparameters Last Checkp	
File Edit View Run Kernel Settings Help		File Edit View Run Kernel Settings Help	
JupyterLab Python [conda env:base] * Trusted		Code	
<pre>Epoch 6/30 368/368 ————— 1s 3ms/step - accuracy: 0.9303 - loss: 0.1618 - val_accuracy: 0.9413 - val_loss: 0.1387 Epoch 7/30 368/368 ————— 1s 3ms/step - accuracy: 0.9338 - loss: 0.1538 - val_accuracy: 0.9433 - val_loss: 0.1318 Epoch 8/30 368/368 ————— 1s 3ms/step - accuracy: 0.9369 - loss: 0.1495 - val_accuracy: 0.9483 - val_loss: 0.1246 Epoch 9/30 368/368 ————— 1s 3ms/step - accuracy: 0.9413 - loss: 0.1395 - val_accuracy: 0.9504 - val_loss: 0.1189 Epoch 10/30 368/368 ————— 1s 3ms/step - accuracy: 0.9422 - loss: 0.1368 - val_accuracy: 0.9527 - val_loss: 0.1143 Epoch 11/30 368/368 ————— 1s 3ms/step - accuracy: 0.9444 - loss: 0.1327 - val_accuracy: 0.9549 - val_loss: 0.1088 Epoch 12/30 368/368 ————— 1s 3ms/step - accuracy: 0.9481 - loss: 0.1253 - val_accuracy: 0.9589 - val_loss: 0.1037 Epoch 13/30 368/368 ————— 1s 3ms/step - accuracy: 0.9501 - loss: 0.1213 - val_accuracy: 0.9600 - val_loss: 0.0993 Epoch 14/30 368/368 ————— 1s 3ms/step - accuracy: 0.9512 - loss: 0.1177 - val_accuracy: 0.9610 - val_loss: 0.0957 Epoch 15/30 368/368 ————— 1s 3ms/step - accuracy: 0.9519 - loss: 0.1157 - val_accuracy: 0.9626 - val_loss: 0.0940 Epoch 16/30 368/368 ————— 1s 3ms/step - accuracy: 0.9541 - loss: 0.1110 - val_accuracy: 0.9667 - val_loss: 0.0874 Epoch 17/30 368/368 ————— 1s 3ms/step - accuracy: 0.9558 - loss: 0.1074 - val_accuracy: 0.9658 - val_loss: 0.0859 Epoch 18/30 368/368 ————— 1s 3ms/step - accuracy: 0.9570 - loss: 0.1045 - val_accuracy: 0.9674 - val_loss: 0.0832 Epoch 19/30 368/368 ————— 1s 3ms/step - accuracy: 0.9585 - loss: 0.1017 - val_accuracy: 0.9680 - val_loss: 0.0816 Epoch 20/30 368/368 ————— 1s 3ms/step - accuracy: 0.9586 - loss: 0.1006 - val_accuracy: 0.9692 - val_loss: 0.0788 Epoch 21/30 368/368 ————— 1s 3ms/step - accuracy: 0.9605 - loss: 0.0966 - val_accuracy: 0.9712 - val_loss: 0.0753 Epoch 22/30 368/368 ————— 1s 3ms/step - accuracy: 0.9612 - loss: 0.0949 - val_accuracy: 0.9715 - val_loss: 0.0748 Epoch 23/30 368/368 ————— 1s 3ms/step - accuracy: 0.9618 - loss: 0.0922 - val_accuracy: 0.9723 - val_loss: 0.0727 Epoch 24/30 368/368 ————— 1s 3ms/step - accuracy: 0.9632 - loss: 0.0893 - val_accuracy: 0.9731 - val_loss: 0.0708 Epoch 25/30 368/368 ————— 1s 3ms/step - accuracy: 0.9643 - loss: 0.0883 - val_accuracy: 0.9734 - val_loss: 0.0668 Epoch 26/30 368/368 ————— 1s 3ms/step - accuracy: 0.9647 - loss: 0.0862 - val_accuracy: 0.9743 - val_loss: 0.0666 Epoch 27/30 368/368 ————— 1s 3ms/step - accuracy: 0.9655 - loss: 0.0846 - val_accuracy: 0.9749 - val_loss: 0.0645 Epoch 28/30 368/368 ————— 1s 3ms/step - accuracy: 0.9659 - loss: 0.0825 - val_accuracy: 0.9749 - val_loss: 0.0622 Epoch 29/30 368/368 ————— 1s 3ms/step - accuracy: 0.9660 - loss: 0.0824 - val_accuracy: 0.9781 - val_loss: 0.0593 Epoch 30/30 368/368 ————— 1s 3ms/step - accuracy: 0.9678 - loss: 0.0789 - val_accuracy: 0.9761 - val_loss: 0.0642</pre>		<pre>Epoch 2/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 3/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 4/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 5/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 6/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 7/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 8/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 9/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 10/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 11/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 12/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 13/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 14/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 15/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 16/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 17/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 18/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 19/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 20/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 21/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 22/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 23/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 24/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan Epoch 25/25 57/57 - 0s - 3ms/step - accuracy: 0.6433 - loss: nan</pre>	
		[531]: <keras.src.callbacks.history.History at 0x1c82a2e7920>	

Deep Learning with Hyperparameters

- Task 2.4 didn't require an image of a confusion matrix.
- However, a print of the new confusion matrix was created.
- The CM displays that Madrid, Basel, Belgrade have some increases pleasant weather days.
- Where in the 2.3 task showed that Madrid and Belgrade were #1 and #2. Basel was #5 in the pleasant weather ranking.



Iteration

- When conducting iterations, I would maybe lean more towards Random Forest and attempt running both the random and grid search.
- I would further break that data down into multiple different decades and look at how the weather has been changing.
- Air Ambulance doesn't have the ability to look at wind speeds so for their most important observations I'd consider sunshine, precipitation and potentially cloud cover.
 - Each of these evaluations could point to a day that is less likely to be raining making it more difficult and riskier to fly.

END